SYSTEM ANALYSIS DOCUMENT (SAD)

UMHackathon 2026

umhackathon@um.edu.my

Project Name: TSRE — Tax-Smart SME Resilience Engine

Version: 1.0

Domain: AI For Economic Empowerment & Decision Intelligence

April 2026

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction

This section introduces the strategic technical solution for the TRSE (Tax Strategist & Receipt Evaluator) system, designed to automate LHDN 2026 e-Invoicing compliance for SMEs.

1.1 Purpose

The system analysis document highlights the technical scope and design-related decisions behind the TRSE development. The key elements of the system covered are:

1. **Architecture:** A cloud-native platform utilizing a Cross-Platform Flutter Client, a Python FastAPI backend, and an AI Service layer consisting of Google Gemini and ZhipuAI (GLM) for OCR and compliance reasoning.
2. **Data Flows:** How receipt images flow from the user's device through the FastAPI backend, get processed by the Vision and RAG agents, and are persistently stored in Supabase (PostgreSQL).
3. **Model Process:** The end-to-end workflow of the core features: uploading a receipt, extracting data via Vision Agent, retrieving tax guidelines via ChromaDB RAG, and evaluating compliance risk using the LLM.
4. **Role of Reference:** This document serves as the technical reference for developers to understand the high-level system components and AI agent dependencies.

1.2 Background

SMEs in Malaysia face a steep learning curve adapting to the mandatory LHDN e-Invoicing rollout.

1. **Previous State:** Manual auditing of receipts by accountants to verify TINs (Tax Identification Numbers), MSIC codes, and SST amounts.
2. **Changes in Major Capabilities:** TRSE introduces an automated "AI Pre-Audit" pipeline. By utilizing a multi-agent LLM approach, the system flags high-risk transactions (e.g., >RM10,000 without a TIN) before they are submitted to LHDN, shielding SMEs from heavy audit fines.

1.3 Target Stakeholder

Stakeholders	Roles	Expectations
SME Owners / Users	Upload receipts, monitor the dashboard for compliance risks, use the AI Chatbar.	Instant compliance verdicts, easy-to-read risk scores, and clear tax-saving recommendations.
LHDN (Indirect)	Regulatory body setting the e-Invoice guidelines.	SMEs submit correctly formatted e-Invoices with valid TINs and MSIC codes.
Development Team	Build the Flutter frontend, FastAPI backend, and AI Agent prompts.	Modular codebase, clean RESTful API contracts, and seamless Supabase integration.
QA Team	Validate AI extraction accuracy and fallback states.	Defined risk scoring logic (e.g., Missing TIN = +40 risk).

2. System Architecture & Design

2.1 High Level Architecture

2.1.1 Overview

Type	Details
System	Cross-Platform App (Flutter Web, iOS, Android)
Architecture	Client-Server with AI Micro-Agents and Supabase BaaS

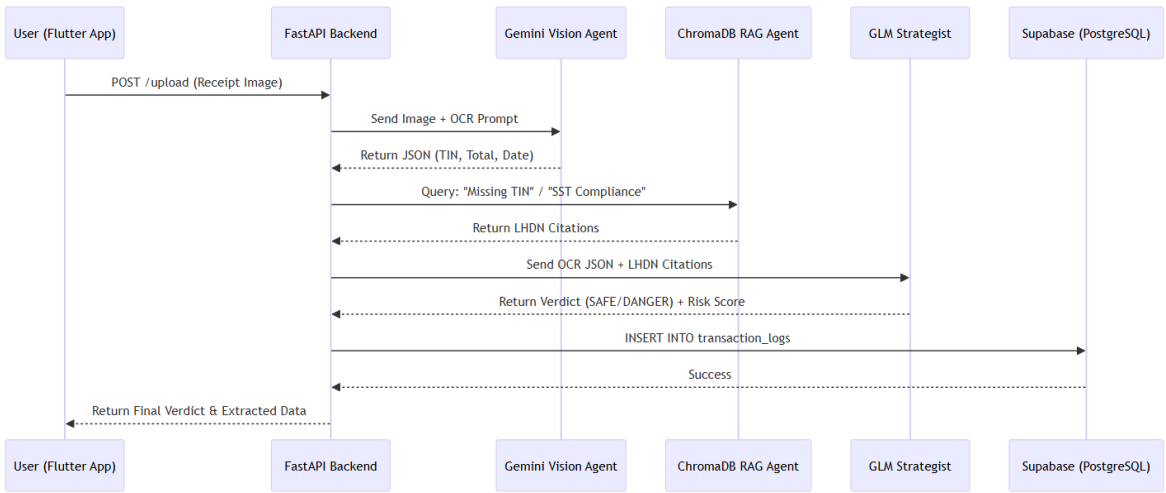
TRSE is structured as a client-server system. The **Flutter Client** handles state management (ValueNotifier) and responsive UI layouts. It communicates via REST API to the **FastAPI Backend**. The backend orchestrates three distinct AI Agents: The Vision Agent (OCR), the RAG Agent (ChromaDB Vector Store), and the GLM Strategist Agent. All transaction histories and user sessions are stored in **Supabase** (PostgreSQL/Auth).

2.1.2 LLM as Service Layer

The architecture integrates LLMs not just as a chatbot, but as the core data-processing engine:

- **Prompt Construction:** The user uploads an image. It is sent to Gemini 2.5 Flash (vision_agent.py) strictly instructed to return JSON (TIN, MSIC, SST).
- **Context Window:** The extracted JSON triggers the rag_agent.py to query ChromaDB (all-MiniLM-L6-v2 embeddings) for LHDN 2026 rules.
- **Parsing & Next Component:** The OCR data and RAG citations are injected into the GLM Strategist context window (glm_prompt.py). The GLM evaluates the data and outputs a structured JSON verdict (SAFE, REVIEW, DANGER).
- **Limitations/Chunking:** Token limits are mitigated by resizing images before upload in main.py (max 2000px width) and forcing strict JSON outputs without markdown fences.

2.1.3 Dependency / Sequence Diagram



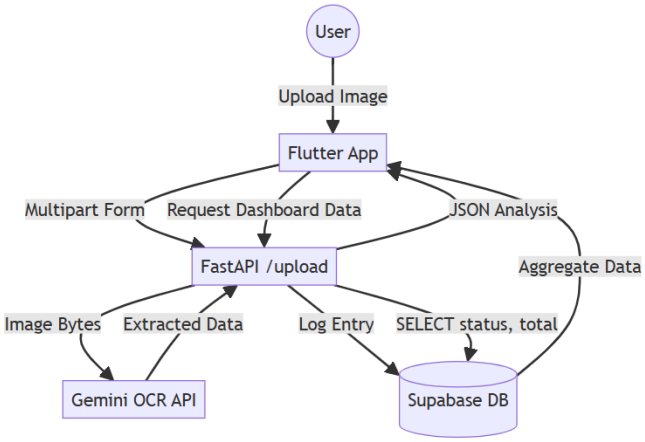
2.2 Technological Stack

Stack	Technology used
Frontend	Flutter & Dart (Cross-platform UI, `ResponsiveLayout`, Supabase Auth)
Backend	Python FastAPI (RESTful API, asynchronous routing, CORS middleware)
Database & Vector Store	Supabase (PostgreSQL for user/transaction logs), ChromaDB (SQLite local vector storage for LHDN documents)

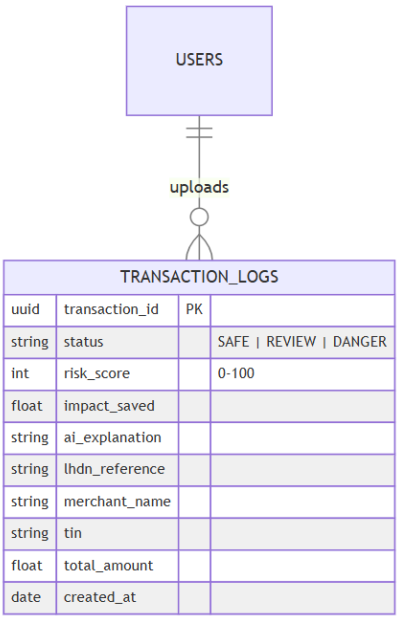
AI Services / Deployment	Google Generative AI (`gemini-2.5-flash`), ZhipuAI (`glm-4v-flash`), HuggingFace sentence-transformers

2.3 Key Data Flows

2.3.1 Data Flow Diagram (DFD)



2.3.2 Normalized Database Schema



3. Functional Requirements & Scope

3.1 Minimum Viable Product (MVP)

Num	Features	Description
1	Automated Receipt OCR Upload	Users upload receipts (Web/Mobile). The system parses TIN, Date, and Total Amount using the Gemini Vision Agent.
2	Compliance Verdict Engine	Flags receipts as SAFE, REVIEW, or DANGER based on LHDN rules (e.g., Missing TIN = +40 risk score).
3	Health Dashboard	Visual aggregate of scanned receipts showing percentage of safe documents and estimated fine exposure.
4	AI Contextual Chat	Users can chat with an AI assistant that retains the context of the most recently scanned receipt to explain LHDN violations.

3.2 Non-Functional Requirements (NFRs)

Quality	Requirements	Implementation
Latency	Image upload and AI analysis must return a result to the user within 5-8 seconds.	Images are dynamically resized in `main.py` before transit to save bandwidth and LLM processing time.
Reliability	Ensure backend doesn't crash on bad AI formatting.	Backend utilizes defensive parsing (Regex stripping of Markdown ```` ```json ```` fences) and strict `try/except` JSON blocks.
Responsiveness	UI must work on Desktop web and Mobile seamlessly.	Implementation of `ResponsiveLayout.dart` utilizing `LayoutBuilder` width constraints (<600px = Mobile).

3.3 Out of Scope / Future Enhancements

- Direct API integration with the official LHDN MyInvois Portal to directly submit the e-Invoices after TSRE validation.

- Multi-tenant agency dashboard for accounting firms managing multiple SMEs.
- Scheduled batch processing for bulk PDF invoice uploads.
- Client-side Privacy Masking to manually masking the private data by dragging a black box in the upload screen
- Authentication allows the system to verify the user or the device to ensure the user can continue or review their previous activity.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1 Technical Evaluation

- Grayscale Rollout & A/B Testing: The LLM prompts (e.g., `SYSTEM_PROMPT` in `glm_prompt.py`) can be A/B tested to see which instructions yield the lowest hallucination rate regarding LHDN tax codes.
- Emergency Fallback: If the ZhipuAI GLM API times out, the backend gracefully degrades by using `gemini-2.5-flash` to handle the final JSON structure, ensuring the user always receives a response.

4.2 Monitoring

- FastAPI console logging is used to track System Error in /upload and Failed to parse Gemini response.
- Supabase built-in dashboard is utilized to monitor Postgres Database row growth and API request limits.

4.3 Assumption

- Users will upload reasonably legible images (supported formats: JPG, PNG, PDF).
- The 2026 LHDN rules (such as RM10,000 threshold and TIN formatting) embedded into the ChromaDB vector store remain accurate for the duration of the system lifecycle.

4.4 External Dependencies

Tools	**Purpose**	**Risks**
Google Gemini API	Primary Vision OCR and Chat agent.	High - Rate limits or token restrictions could pause the primary extraction flow.
Supabase	Cloud Postgres Database and user Authentication.	Low - Highly scalable, but reliant on external cloud uptime.
ZhipuAI (GLM API)	Advanced multi-step reasoning	Medium - Network latency

	for compliance.	for international API calls could impact the 5s SLA.

5. Project Management & Team Contributions

5.1 Project Timeline

Timeline	Events
Day1-2	Project Planning, Architecture setup, FastAPI routing
Day3-5	RAG/ChromaDB ingestion of LHDN rules.
Day5-9	LangGraph agent orchestration and gemini prompt engineering, Next.js frontend development
Day9-10	bug fixing
Day10-11	Preparation of PRD, Preparation of SAD, QA testing with synthetic data, and Pitch Video recording.

5.2 Team Members Role

Priority	Feature / Area	Member 1: Lee	Member 2: Gan
Level 1: Foundation (DO FIRST)	Dev Environment & Data	Setup GitHub & DB: Initialize repo and set up the PostgreSQL/Supabase schema.	Frontend Skeleton: Build the base Flutter/Next.js UI with the image upload picker.

Level 2: The Core Agents	Backend & AI Engine	FastAPI Ingestion: Build the secure endpoint to receive and sanitize receipt images.	OCR & RAG Setup: Seed ChromaDB with LHDN rules and test the Vision Agent extraction.
Level 3: Intelligence & Reasoning	Decision Logic	Audit Trail Logic: Code the function to persistently store compliance logs in SQL.	Master Strategist: Write the gemini prompt to cross-reference Vision + RAG outputs for a verdict.
Level 4: User Experience	Frontend Integration	API Documentation: Define the JSON contract for the "Compliance Card" output.	Compliance Card UI: Link the real GLM output to the Green/Amber/Red status card.
Level 5: Impact & Polish	Final Submission	System Analysis (SAD): Finalize technical diagrams (DFD/ERD) to match the code.	Pitch & Demo: Record the video and finalize the PRD for submission.

-

5.3 Recommendations

- Scalability: To reduce API costs and lower latency, implement a caching layer (e.g., Redis) for the /ai-chat queries if users ask identical questions about general LHDN rules.
- Performance: Move the heavy LLM processing from a synchronous HTTP request to an asynchronous Background Task (using Celery or FastAPI BackgroundTasks) combined with WebSockets in Flutter to show a live processing status.